



Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS132 - Imperative Programming

Copyright ©2026 by the 2026 staff of COS 132. All rights reserved.

Practical 5: Fixing Broken Code

Release Date: 13-04-2026 at 06:00

Due Date: 17-04-2026 at 23:59

Late Deadline: 19-04-2026 at 23:59

Total Marks: 94

**Read the entire specification before starting
with the practical.**

Contents

1	General Instructions	3
2	Overview	4
3	Background	4
3.1	Mark Breakdown	4
4	Your Task:	4
4.1	Kitchen	5
4.1.1	Variables	5
4.1.1.1	const int id = 1	5
4.1.1.2	const int size = 70	5
4.1.1.3	const double obstacles = 0.8	5
4.2	LivingRoom	5
4.2.1	Variables	5
4.3	Bathroom	5
4.3.1	Variables	5
4.4	BedroomOne	5
4.4.1	Variables	5
4.5	BedroomTwo	6
4.5.1	Variables	6
4.6	Bug	6
4.6.1	Functions	6
4.6.1.1	std::string getBugInfo(const std::string bugType)	6
4.6.1.2	int getBugDensity(const std::string bugType)	7
4.6.1.3	bool isSevereInfestation(const std::string bugType, int count)	7
4.7	House	7
4.7.1	Functions	7
4.7.1.1	double roomObstacles(int id)	7
4.7.1.2	int roomSize(int id)	7
4.7.1.3	int getEffectiveArea(double houseSize)	8
4.7.1.4	bool canFumigateRoom(int id)	8
4.7.1.5	int countAccessibleRooms()	8
4.7.1.6	int fumigate(double solution)	8
4.8	Fumigator	9
4.8.1	Functions	9
4.8.1.1	double calculateSolution(int effectiveArea, int bugDensity, bool severe)	9
4.8.1.2	int estimateTreatmentTime(int accessibleRooms, bool rushJob)	9
4.8.1.3	bool startFumigation(std::string bugType, int count, int houseSize, bool rushJob)	9
5	Implementation Details	9
6	Upload Checklist	10
7	Submission	10

1 General Instructions

- *Read the entire assignment thoroughly before you begin coding.*
- This assignment should be completed individually.
- Every submission will be inspected with the help of dedicated plagiarism detection software.
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 48 hours after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at <https://portal.cs.up.ac.za/files/departamental-guide/>.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

In this practical you will be fixing broken code (finding bugs) by rectifying a house fumigation simulator thus finding bugs, by finding bugs!

3 Background

Bug detection and correction are fundamental aspects of software development. Undetected bugs can lead to incorrect program behaviour, system crashes and unreliable outputs, which may have significant consequences depending on the application domain. In critical systems, such as those used in healthcare, finance or engineering, even minor errors can result in severe failures.

Debugging not only improves the correctness of a program but also enhances its efficiency, maintainability, and robustness. It encourages developers to think critically about program logic, test edge cases and understand how different components of a system interact.

3.1 Mark Breakdown

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
Bug	27
House	45
Fumigate	22
Total	94

4 Your Task:

You are required to rectify all of the functions laid out in this section such that they work as explained in the relevant sub-section. Remember that the point is to rectify the code, not reimplement it and that there will be **intentional** logical and syntax errors within the code.

The namespaces described in this section are used to store different constant values pertaining to the different rooms of a house. There are only 5 rooms in any given house. **The variable descriptions under Kitchen apply to all the other namespaces.**
These namespaces must be in the House.h file.

4.1 Kitchen

4.1.1 Variables

4.1.1.1 `const int id = 1`

- The unique identifier of the room.

4.1.1.2 `const int size = 70`

- The size of the room in square meters (m^2).

4.1.1.3 `const double obstacles = 0.8`

- The proportion of the room that is obstructed by furniture, be it built-in fixtures or free-standing furniture. (i.e. 80% of the Kitchen is occupied by furniture).

4.2 LivingRoom

4.2.1 Variables

- `const int id = 2`
- `const int size = 100`
- `const double obstacles = 0.15`

4.3 Bathroom

4.3.1 Variables

- `const int id = 3`
- `const int size = 25`
- `const double obstacles = 0.4`

4.4 BedroomOne

4.4.1 Variables

- `const int id = 4`
- `const int size = 60`
- `const double obstacles = 0.6`

4.5 BedroomTwo

4.5.1 Variables

- `const int id = 5`
- `const int size = 40`
- `const double obstacles = 0.2`

4.6 Bug

The following functions are implemented for you in the `Bug.cpp` file. You will have to first create the corresponding `Bug.h` file and then rectify these functions inside the `Bug.cpp` file such that they work as intended. Remember to ensure that your naming is correct.

4.6.1 Functions

4.6.1.1 `std::string getBugInfo(const std::string bugType)`

- Returns an English description of the given bug type. The `_`(underscore) indicates **one space**. Any other whitespace is ignored (**Space at the end of a sentence is not a newline**)
- Types of “bugs” and their descriptions:
 - cockroach:

Listing 1: Cockroach Info Format

```
Nocturnal_and_highly_adaptable_savengers .  
_They_tend_to_cluster_together_in_dark ,_moist_areas .
```

Listing 1: Cockroach Info Format

- ant:

Listing 2: Ant Info Format

```
Small ,_social_insects_known_for_being_diligent_workers .  
_Extremely_high_density .
```

Listing 2: Ant Info Format

- bedbug:

Listing 3: Bedbug Info Format

```
Small ,_flat ,_parasitic_insects_that_feed_on_the_blood_of_humans_and_  
animals ._They_are_nocturnal_and_can_hide_in_small_crevices .
```

Listing 3: Bedbug Info Format

- any unknown type:

Listing 4: Unknown Bug Type Info Format

```
Unknown_bug_type._No_information_available.
```

Listing 4: Unknown Bug Type Info Format

4.6.1.2 `int getBugDensity(const std::string bugType)`

- Returns the infestation density for the given bug type. (i.e. Per 1 ant that you see, there are usually $\times 80$ more.)
- Density values:
 - cockroach = 40
 - ant = 80
 - bedbug = 30
 - unknown = 0

4.6.1.3 `bool isSevereInfestation(const std::string bugType, int count)`

- Determines whether a given infestation is severe based on bug type and count.
- Calculates `count  $\times$  getBugDensity` and returns `true` if the result is above 500, otherwise `false`.
- The `count` is just a rough number of said bug seen, based on it, an estimated total number is calculated using the typical bug density.

4.7 House

The following functions are implemented for you in the `House.cpp` file. You will have to first create the corresponding `House.h` file and then rectify these functions inside the `House.cpp` file such that they work as intended. Remember to ensure that your naming is correct.

4.7.1 Functions

4.7.1.1 `double roomObstacles(int id)`

- This function simply returns the obstacle value of the room corresponding to the passed-in `id`.
- If the `id` does not exist, return 0.0.

4.7.1.2 `int roomSize(int id)`

- This function simply returns the size value of the room corresponding to the passed-in `id`.
- If the `id` does not exist, return 0.

4.7.1.3 `int getEffectiveArea(double houseSize)`

- Determines the area of the house that can be fumigated after accounting for obstacles.
- The effective area starts off equal to the `houseSize` in square meters (m^2)
- For each room, the area decreases by $(\text{roomSize} \times \text{roomObstacles})$ rounded to the nearest integer.
- If the `houseSize` is less than or if the `houseSize` is less than the sizes of all the rooms combined, return 0. Additionally, the final effective area cannot go below 0.
- *Note: `houseSize` can be greater than the total size of all the rooms combined as it include passageways, which may be of various sizes.*

4.7.1.4 `bool canFumigateRoom(int id)`

- This function determines whether a room can be physically fumigated given its obstacle percentage.
- It first looks up the room's size using the `roomSize` function and object density using `roomObstacle`.
- The room is checked every 10 m^2 , thus the number of iterations will be how many times 10 fits into the room's size. *Hint: Think about integer division.*
- Initially, the amount of `clearSpace` is the size of the room and it decreases by 10 at each iteration if the room's object density is above 0.4.
- If `clearSpace` becomes negative, it is reset to 0.
- Returns `true` if the final `clearSpace` exceeds 20, otherwise `false`.

4.7.1.5 `int countAccessibleRooms()`

- Counts how many rooms can be fumigated by calling `canFumigateRoom()` for each room and incrementing the total count on if the room can be fumigated.

4.7.1.6 `int fumigate(double solution)`

- In this function we traverse the house and fumigate all rooms that can be fumigated.
- The rooms are visited sequentially from 1 to 5 as long as there is still some fumigation solution.
- Every room uses up $0.4 \times \text{roomSize}$ amount of solution.
- It returns the number of successfully fumigated rooms.

4.8 Fumigator

The following functions are implemented for you in the `Fumigator.cpp` file. You will have to first create the corresponding `Fumigator.h` file and then rectify these functions inside the `Fumigator.cpp` file such that they work as intended. Remember to ensure that your naming is correct.

4.8.1 Functions

4.8.1.1 `double calculateSolution(int effectiveArea, int bugDensity, bool severe)`

- Calculates the volume (in ml) of chemical solution required for treatment.
- The base amount is computed as $\text{effectiveArea} \times \text{bugDensity} \times 0.75$.
- If the infestation is severe, the base amount is further increase by a rate of 1.5.

4.8.1.2 `int estimateTreatmentTime(int accessibleRooms, bool rushJob)`

- Estimates the total fumigation time in minutes.
- The base time is the value of $\text{accessibleRooms} \times 30$ (about 30 minutes per room).
- If `rushJob` is `true`, the time is multiplied by 0.75 to cut it down by 25%, then rounded to the nearest integer.

4.8.1.3 `bool startFumigation(std::string bugType, int count, int houseSize, bool rushJob)`

- This function is the entry point of the fumigation process.
- It calculates all the values required by `fumigate` and calls the function.
- As `win isSevereInfestation`, `count` is just a rough number of said bug seen.
- Returns `true` if the number of fumigated rooms is at least 3 and the actual time taken is less than or equal to the time estimated otherwise it returns `false`.
- *Note: The actual time taken can be calculated by calling `estimateTreatmentTime` with the number of rooms that were successfully fumigated and `rushJob`.*

5 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use `c++98`.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- Do not use `namespace std` in any of the files.

- You may only use the following libraries:
 - `<iostream>`
 - `<cmath>`
 - `<string>`
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment.

6 Upload Checklist

The following c++ files should be in a zip archive named `uXXXXXXXXX.zip` where `XXXXXXXXX` is your student number:

- `Bug.cpp`
- `House.cpp`
- `Fumigator.cpp`
- `main.cpp`
- Any textfiles used by your `main.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

7 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (<https://ff.cs.up.ac.za/>). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

1

and run with the following command:

```
./main
```

1

Remember your `h` file will be overwritten, so ensure you do not alter the provided `h` files.

You have 5 submissions, and your best mark will be your final mark. Upload your archive to the Practical 5 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**

Remember, bugs can hide ;)